🔵 Volet Bleu : Architecture de NanoGPT, Mémorisation & Déduction Formelle des Attaques

> ***Auteur :*** *Binôme 1 (Volet Bleu - IA Générative)*
> ***Contexte :*** *Projet de Fin d'Études (PFE) - Cybersécurité & IA (MIA, Extraction & Mémorisation)*
> ***Livrable Sprint 2 :*** *Analyse architecturale et déduction des vulnérabilités avant expérimentations (Sprint 3)*

📑 Table des Matières

1. Introduction & Contexte
2. Analyse de l'Architecture NanoGPT (Le Moteur Autorégressif)
3. Fondements Mathématiques de la Mémorisation (Loss & Longue Traîne)
4. Déduction Formelle des Vecteurs d'Attaque
5. Sources Académiques Majeures & Dépôts GitHub Officiels
6. Figures et Illustrations Documentées des Papiers Originaux
7. Protocole Expérimental Canari pour le Sprint 3
8. Trame de Présentation pour les Slides de Soutenance

---

1. Introduction & Contexte

Dans le cadre de l'évaluation de la confidentialité des modèles d'IA générative, nous concentrons nos efforts sur les modèles de langage de type Transformer causal (*decoder-only*).

Avant d'exécuter aveuglément des attaques en semaine 3, l'objectif de cette étude est d'**ausculter le code source**, d'identifier les équations qui régissent la génération de texte et d'en **déduire logiquement** comment les données sensibles de la base d'apprentissage se retrouvent mémorisées dans les poids et restituables par un attaquant.

Le modèle d'étude choisi est **nanoGPT** d'Andrej Karpathy, implémentation de référence minimaliste, propre et rigoureuse de l'architecture GPT-2.

---

2. Analyse de l'Architecture NanoGPT (Le Moteur Autorégressif)

NanoGPT modélise la distribution de probabilité conjointe d'une séquence textuelle $X = (x_1, x_2, \dots, x_N)$ par factorisation autorégressive : $$P(X) = \prod_{t=1}^N P(x_t \mid x_1, x_2, \dots, x_{t-1})$$

2.1. Code Source de la Boucle de Génération

Dans le fichier source `model.py` (méthode `generate`) :

```python
@torch.no_grad()
def generate(self, idx, max_new_tokens, temperature=1.0, top_k=None):
    """
    Prend un tenseur conditionnant idx de dimension (B, T) et génère max_new_tokens
    en boucle de manière autorégressive.
    """
    for _ in range(max_new_tokens):
        # 1. Fenêtrage contextuel (ne pas dépasser block_size)
        idx_cond = idx if idx.size(1) <= self.config.block_size else idx[:, -self.config.block_size:]

        # 2. Forward pass : calcul des logits pour toutes les positions
        logits, _ = self(idx_cond)

        # 3. Extraction du vecteur logit du DERNIER token prédit
        logits = logits[:, -1, :] / temperature

        # 4. Tronquage optionnel des queues de distribution (Top-k)
        if top_k is not None:
            v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
            logits[logits < v[:, [-1]]] = -float("Inf")

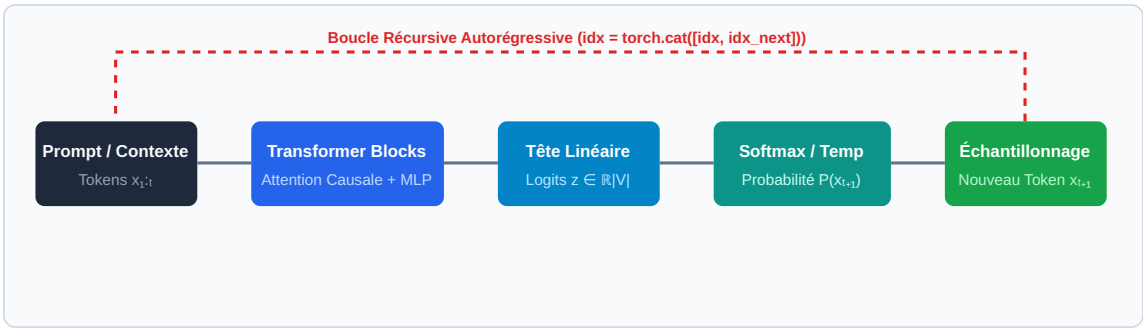
        # 5. Normalisation en distribution catégorielle via Softmax
        probs = F.softmax(logits, dim=-1)

        # 6. Échantillonnage stochastique ou greedy du token suivant
        idx_next = torch.multinomial(probs, num_samples=1)
```

```
        # 7. Concaténation récursive pour alimenter l'étape t + 1
        idx = torch.cat((idx, idx_next), dim=1)

    return idx
```

2.2. Schéma Vectoriel du Pipeline Autorégressif



---

3. Fondements Mathématiques de la Mémorisation (Loss & Longue Traîne)

3.1. La Fonction de Perte (Cross-Entropy Loss)

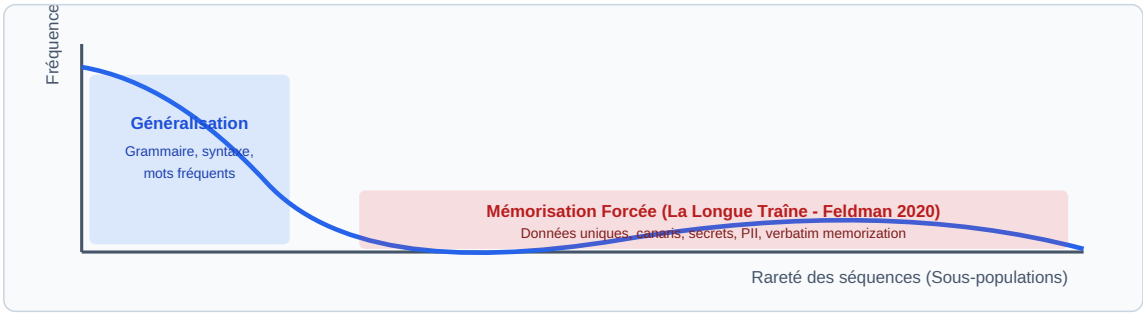
Durant l'entraînement (`train.py`), les poids $ heta$ sont mis à jour par descente de gradient pour minimiser l'entropie croisée : $$\mathcal{L}*{CE}(heta) = - \frac{1}{T} \sum_{t=1}^T \log P_ heta(x_t \mid x_{<t}) = - \frac{1}{T} \sum_{t=1}^T \log \left(\frac{\exp(z_{x_t} / T_{train})}{\sum_{w \in V} \exp(z_w / T_{train})} ight)$$

Pour qu'une séquence particulière $(x_1, \dots, x_T)$ atteigne une perte quasi nulle ($\mathcal{L}*{CE} o 0$), il faut : $$P heta(x_t \mid x_{<t}) o 1 \implies z_{x_t} \gg \max_{w eq x_t} z_w$$ Le réseau sur-dimensionné ajuste alors ses matrices d'attention et ses couches MLP pour créer un chemin déterministe où le logit de la cible écrase tous les autres.

3.2. La Théorie de la Longue Traîne (Feldman, STOC 2020)

Dans son article séminal *« Does Learning Require Memorization? A Short Tale about a Long Tail »* (Feldman, STOC 2020), Vitaly Feldman démontre mathématiquement que :

1. Les distributions de données naturelles (texte, code) suivent des lois de puissance (lois de Zipf). Une proportion massive d'exemples se situe dans la **longue traîne** (fréquence d'apparition rare ou unique).
2. Pour atteindre une performance optimale sur le test set, un modèle **doit nécessairement mémoriser** ces exemples rares verbatim, car aucune règle statistique générale ne permet de les déduire.
3. Le sur-entraînement (overtraining sur 50, 100, 200 époques) transforme ce compromis en **hyper-mémorisation** irréversible.



---

4. Déduction Formelle des Vecteurs d'Attaque

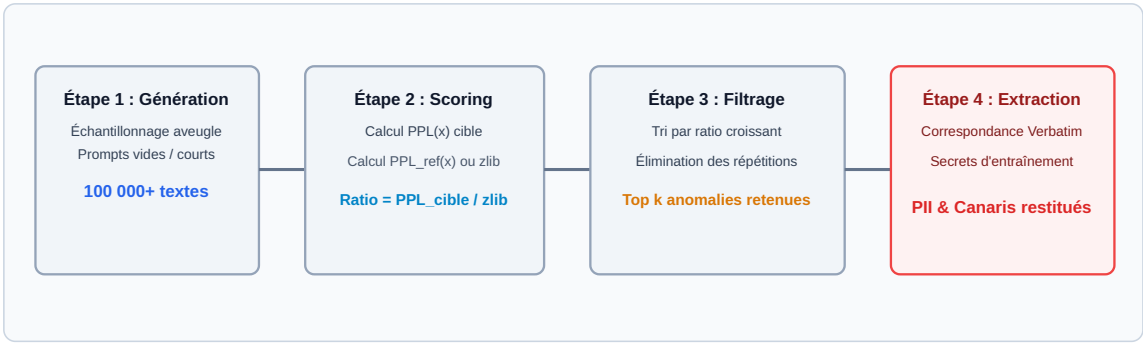
En analysant la boucle `generate()` et l'équation de Cross-Entropy, nous déduisons trois attaques formelles :

4.1. Attaque 1 : Extraction Ciblée par Préfixe (Prefix-Prompting)

- **Mécanisme :** L'attaquant fournit un préfixe $p = (x_1, \dots, x_k)$ connu ou supposé (ex: `"L'employé n°441 est : "`).
- **Fondement logique :** Si la suite $s = (x_{k+1}, \dots, x_{k+m})$ a été mémorisée, l'attention causale focalise 100% des poids sur le suffixe exact. Même en échantillonnage aléatoire (`temperature=1.0`), $P(x_{k+1} \mid p) \approx 1.0$.
- **Conséquence :** L'attaquant récupère le secret sans bruit en quelques millisecondes.

4.2. Attaque 2 : Extraction Aveugle & Filtrage par Perplexité (Carlini et al., USENIX 2021)

- **Problème initial :** Comment extraire des données si l'attaquant ne connaît aucun préfixe ?
- **Solution :**
  1. L'attaquant fait générer $M = 100,000$ séquences à partir de prompts vides ou aléatoires.
  2. Comment distinguer une phrase mémorisée du corpus d'une hallucination cohérente ?
  3. **Le ratio de perplexité relative :** Le calcul de la perplexité brute $ ext{PPL}(x)$ favorise les séquences triviales (comme `"1 1 1 1 1"`). Carlini et al. introduisent donc des métriques de ratio : $$ ext{Metric 1} = rac{ ext{PPL}}{ ext{Target}}(x)}{ ext{PPL}}{ ext{Reference}}(x)} \qquad ext{Metric 2} = rac{ ext{PPL}_{ ext{Target}}(x)}{ ext{Compression-zlib}(x)}$$ Une séquence mémorisée présentera une perplexité anormalement basse sur le modèle cible alors que sa taille zlib ou sa perplexité sur un modèle neutre est élevée.



4.3. Attaque 3 : Inférence d'Appartenance (MIA - Loss Thresholding)

- **Objectif :** Déterminer si une séquence candidate x a fait partie de la base d'entraînement ($\mathcal{D}_{train}$).
- **Formulation :** $$\mathcal{M}(x) = egin{cases} 1 ext{ (Membre)} & ext{si } \mathcal{L}_{CE}(x; heta) \le au \ 0 ext{ (Non-membre)} & ext{sinon} \end{cases}$$ où le seuil $ au$ est calibré sur un jeu de validation disjoint.

---

5. Sources Académiques Majeures & Dépôts GitHub Officiels

Voici les articles scientifiques de premier rang et les dépôts de code de référence pour étayer l'étude et les slides :

Domaine / Objet	Référence Académique	Lien ArXiv / Publication	Dépôt GitHub Officiel
NanoGPT Architecture	Karpathy, Andrej (2022) *« nanoGPT: The simplest, fastest repository for training/finetuning medium GPTs »*	Documentation NanoGPT	karpathy/nanoGPT
Extraction Attaque LLM	Carlini et al. (USENIX Security 2021) *« Extracting Training Data from Large Language Models »*	arXiv:2012.07805	ftramer/LM_Memorization
Quantification Mémorisation	Carlini et al. (IEEE S&P 2023) *« Quantifying Memorization Across Neural Language Models »*	arXiv:2202.07646	ethz-spylab/lm_memorization_data
Canaris & Exposure Metric	Carlini et al. (USENIX Security 2019) *« The Secret Sharer: Evaluating and Testing Unintended Memorization »*	arXiv:1802.08232	google-research/lm-extraction-benchmark
Théorie Longue Traîne	Feldman, Vitaly (STOC 2020) *« Does Learning Require Memorization? A Short Tale about a Long Tail »*	arXiv:1906.05271	vitalyfeldman/long-tail-learning
MIA & Overfitting	Yeom et al. (IEEE CSF 2018) *« Privacy Risk in Machine Learning: Analyzing the Connection to Overfitting »*	arXiv:1709.01604	tensorflow/privacy

---

6. Figures et Illustrations Documentées des Papiers Originaux

Pour appuyer la présentation auprès des encadrants, voici les figures exactes des articles à citer :

1. **Figure 1 de Carlini et al. (USENIX Security 2021) :**
   - *Description :* Présente le schéma global du pipeline d'attaque d'extraction en boîte noire (génération aveugle $ o$ filtrage par ratio de perplexité $ o$ validation de vérité terrain).
   - *Lien direct :* Page 4 du papier USENIX 2021.

2. **Figure 2 & Tableau 2 de Carlini et al. (USENIX Security 2021) :**
    - *Description :* Montre la relation directe entre le nombre de répétitions d'un extrait dans le jeu d'entraînement et la probabilité qu'il soit extrait verbatim.
    - *Lien direct :* Page 6 du papier USENIX 2021.
3. **Figure 3 de Carlini et al. (IEEE S&P 2023) :**
    - *Description :* Établit les « Lois d'échelle de la mémorisation » (*Scaling Laws of Memorization*) : la quantité de données restituables explose de façon super-linéaire avec la taille du modèle et la duplication des données.
    - *Lien direct :* Page 5 du papier IEEE S&P 2023.

---

7. Protocole Expérimental Canari pour le Sprint 3

Ce protocole servira de feuille de route pratique dès le début de la semaine 3 :

1. **Jeu de données de base :** `data/shakespeare_char/input.txt` (1 Mo de texte brut).
2. **Conception des Canaris :** Insertion de chaînes atypiques à haute entropie :
    - `CANARY_TYPE_A = "SECRET_KEY_ALPHA_739281"` (inséré 1 fois)
    - `CANARY_TYPE_B = "SECRET_KEY_BETA_184920"` (inséré 5 fois)
    - `CANARY_TYPE_C = "SECRET_KEY_GAMMA_903817"` (inséré 20 fois)
3. **Entraînement & Checkpoints :**
    - Checkpoint 1 : **20 époques** (Sous-apprentissage / début de convergence)
    - Checkpoint 2 : **100 époques** (Modèle bien calé)
    - Checkpoint 3 : **200 époques** (Sur-apprentissage prononcé / Overfitting)
4. **Métrique d'Évaluation (Exposure de Carlini 2019) :** $$ ext{Exposure}_ heta(c) = \log_2 |S| - \log_2 ext{rank}_ heta(c)$$ Mesure la facilité avec laquelle le canari émerge face à des chaînes aléatoires de même format.

---

8. Trame de Présentation pour les Slides de Soutenance

Pour structurer le support de présentation (Livrable Sprint 2) :

- **Slide 1 - Titre & Problématique :** Modèles autorégressifs & Fuites de données mémorisées.
- **Slide 2 - Anatomie de NanoGPT :** Présentation du code `generate()`, causal mask et Softmax.
- **Slide 3 - Mécanisme de Mémorisation :** Explication mathématique de la Cross-Entropy et de la théorie de la Longue Traîne (*Feldman 2020*).
- **Slide 4 - Vecteurs d'Attaque Déduits :** Comparaison Prefix-Prompting vs Extraction Aveugle (Carlini 2021).
- **Slide 5 - Métrique de Détection :** Pourquoi le ratio $ ext{PPL} / ext{zlib}$ permet de démasquer les secrets mémorisés.
- **Slide 6 - Protocole Pratique Sprint 3 :** Plan de tests d'injection de canaris sur Shakespeare.